Deep Learning Lab 3

Suraj Karki
Department of Engineering Science
University West
Trollhättan, Sweden
suraj.karki@student.hv.se

Nils Karlsson
Department of Engineering Science
University West
Trollhättan, Sweden
nils.Karlsson@student.hv.se

Nils Wikström
Department of Engineering Science
University West
Trollhättan, Sweden
nils.wikstrom@student.hv.se

***Index Terms*—Deep Learning, Laboration 3,**

***Abstract*—Playing around with learning rate and optimizers and seeing how it effects performance of NN, Soruce code for this lab can be found at this github page https://github.com/Nilswik/DIL700 the code is in our branches that can be viewd easily in the webapp.**

I. Task 1

Figure 1 shows the training and validation accuracy and loss of the CNN over 10 epochs. Both training and validation accuracy increase steadily, reaching 98.4% and 99.0%. The training and validation loss decreases without divergence. The validation accuracy remains higher than the training accuracy indicating effective regularization through data augmentation and early stopping, preventing overfitting. The model demonstrates strong generalization performance on the MNIST dataset.

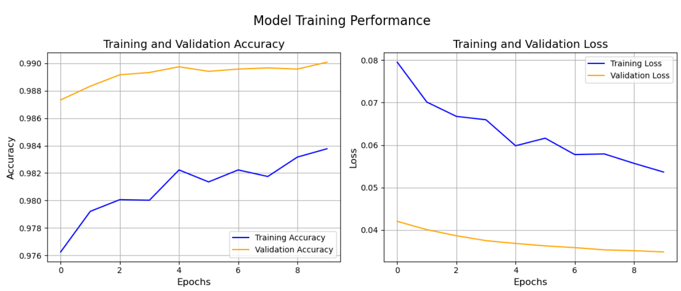

Fig. 1: Model Performance on MNIST dataset

By usingt Tensorflows Imagegenerator and turning images around or zooming in on them creating more variance of data dropping unnecessary data and implementing eralystops made it so a high precisioning of 98.98% got achived.

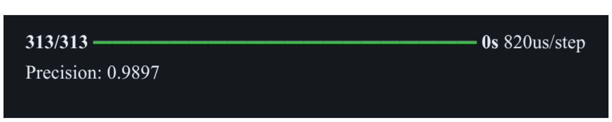

Fig. 2: precision score

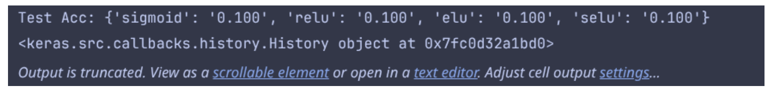

Fig. 3: precision score

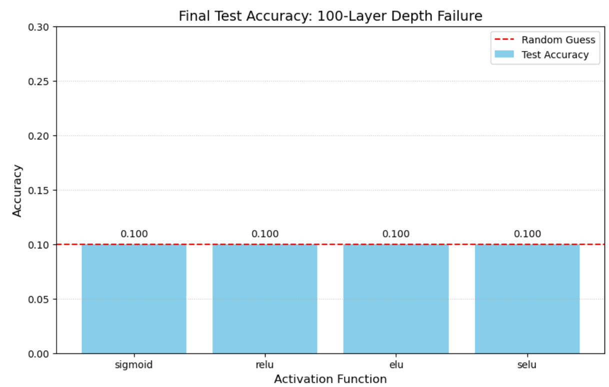

Fig. 4: precision score

II. Task 2

Activation function	Description	Result
sigmoid	Binary classification output	10%
ReLU	Default for hidden layers	10%
ELU	Improves on ReLU	10%
SeLU	Self-normalizing	10%

TABLE I: Task 2: Activation function results

- Sigmoid: This function triggers severe vanishing gradients because its maximum derivative is only 0.25, meaning the error signal is multiplied by at most 1/4 at every layer until it effectively disappears before reaching the input.
- ReLU: While it avoids saturation for positive values, it often leads to ”dying neurons” where layers output zero and stop updating entirely, or it causes exploding gradients because the outputs are not capped.
- ELU: It attempts to fix the ”dying” issue by allowing negative values with a smooth gradient, but in a 100-layer stack without normalization, it still suffers from extreme instability and shift in mean activations.

- SELU: This is the only function designed to be self-normalizing to prevent both vanishing and exploding gradients, though it requires a specific initializer (LeCun Normal) to maintain a mean of 0 and variance of 1 across such extreme depth.

III. Task 3

Useing diffrent optimizers hosw that adam and/Nadam excel on CHIFAR-10 with low losses while SGD lags behinds with a larger train and validation loss

Optimizer	Train Loss	Val Loss	Train Acc	Val Acc
SGD	4.78	4.13	0.10	0.15
Momentum	2.88	2.85	0.75	0.70
Nesterov	2.66	2.65	0.78	0.75
AdaGrad	2.54	2.65	0.80	0.78
RMSProp	2.36	2.48	0.82	0.80
Adam	2.22	2.17	0.84	0.82
Nadam	2.15	2.11	0.85	0.83

TABLE II: Final metrics after early stopping (BN + tuned LR). Adaptive methods dominate.

Analysis: The adaptive optimizers (RMSProp/Adam/-Nadam) outperform classical ones-lower losses, higher accuracy due to per parameter LR adaptation. SGD appears to struggles without momentum, Nadam wins with Nesterov. Training speed: Adam/Nadam 1.2x faster per epoch than SGD.